

ENTERPRISE TECHNICAL DOCUMENTATION

Masar

مسار

Dubai Commute Intelligence Platform

Congestion Prediction · Multi-Agent Route Reasoning · Bilingual Commute Guidance

Document Type:	Technical Architecture & Engineering Reference
Repository:	MohdSiddiq12/Masar (branch: testing)
Version:	1.0
Status:	Active Development — Working Prototype
Prepared For:	Engineering Review, Portfolio & Academic Submission
Generated:	September 10, 2026

Core Stack

LangGraph · XGBoost · NetworkX · Supabase (Postgres) · Groq LLM · TomTom API · OpenWeatherMap API · Docker ·
Render · GitHub Actions

Version History

Version	Date	Author	Summary of Changes
1.0	2026-09-10	Documentation Generator	Initial enterprise-grade technical documentation compiled from the project README, source tree, and component inventory.

This document is generated directly from the current state of the Masar repository (branch: testing). It reflects source-verified facts, explicitly documented assumptions, and forward-looking recommendations. No functions, files, or APIs are described beyond what exists in the project's own documentation.

Table of Contents

This table of contents is generated from document headings. In Microsoft Word, right-click anywhere inside it and choose “Update Field” (or press F9) if page numbers do not appear immediately after opening.

Version History	2
Table of Contents.....	3
1. Executive Summary	7
1.1 Objectives	7
1.2 Target Users & Stakeholders	7
1.3 Success Criteria	7
1.4 Scope Summary	8
2. Project Overview & Scope	9
2.1 Core Feature Set	9
2.2 Repository Root Structure	9
2.3 Folder Responsibilities	10
3. Business Problem & Solution	12
3.1 The Problem.....	12
3.2 The Solution	12
4. Functional & Non-Functional Requirements	13
4.1 Functional Requirements.....	13
4.2 Non-Functional Requirements.....	13
5. High-Level System Architecture.....	14
5.1 Runtime Flow (Textual).....	14
6. Software Architecture (Layered View).....	15
7. Workspace & Directory Structure — Detail.....	16
8. Complete File-by-File Documentation.....	17
9. Technology Stack & Dependency Report	18
9.1 Runtime Environment.....	18
9.2 Dependency Inventory	18
9.3 Runtime Requirements.....	18
9.4 Environment Variables	19
9.5 Secrets & Feature Flags	19
10. AI Agents & Model Architecture.....	20
10.1 Failure Behavior by Agent.....	21
10.2 Congestion Classifier (XGBoost).....	22
10.3 LLM (Groq)	22
11. Data Pipeline — Live Traffic & Weather Ingestion	23

11.1 Source Tables	23
11.2 Feature Schema for Training.....	24
11.3 Data Cleaning & Validation	24
11.4 Train / Validation / Test Split	24
12. LangGraph Workflow & Component-Level Design	25
masar/nodes/predictor.py	25
masar/nodes/router.py	25
masar/nodes/context.py	25
masar/nodes/optimizer.py	25
masar/nodes/synthesis.py.....	26
masar/live_data.py.....	26
masar/live_traffic.py.....	26
13. Model Training & MLOps Pipeline	28
13.1 Reproducibility & Versioning	29
13.2 Training Artifacts.....	29
13.3 Current Benchmark (Synthetic Data).....	29
14. Function-Level API Documentation	31
14.1 train_model.py	31
14.2 masar/nodes/predictor.py	31
14.3 masar/nodes/router.py	32
14.4 masar/nodes/context.py	32
14.5 masar/nodes/optimizer.py	32
14.6 masar/nodes/synthesis.py.....	33
14.7 web_app.py	33
14.8 masar/live_data.py	34
15. Backend API — Input / Output Specification.....	35
15.1 GET /health	35
15.2 GET /	35
15.3 GET /api/traffic	35
15.4 GET /api/locations	35
15.5 GET /api/traffic/latest?location=Marina	35
15.6 POST /api/recommend	35
15.7 API Validation & Status Handling.....	36
16. Configuration & Environment Guide	37
16.1 Configuration Sources.....	37
16.2 Variable Catalog.....	37
16.3 Secrets Management.....	37

17. External Services & Integrations.....	38
17.1 TomTom.....	38
17.2 OpenWeatherMap.....	38
17.3 Supabase.....	38
17.4 Rate Limits & Retries.....	38
18. Deployment Architecture & Guide	39
18.1 Local Development	39
18.2 Docker	39
18.3 Production Guidance	39
19. Execution Lifecycle.....	41
19.1 Startup Sequence.....	41
19.2 Inference Sequence	41
20. Performance Analysis	43
20.1 Built-in Observability	43
20.2 Current Monitoring Gaps.....	43
20.3 Recommended Future Monitoring	43
20.4 Performance Bottlenecks & Scaling Limits	43
21. Error Handling Strategy	44
22. Security & Compliance.....	45
22.1 Data Protection Recommendations	45
22.2 Compliance Notes.....	45
23. Current Limitations.....	46
23.1 Technical Limitations	46
23.2 Data Limitations.....	46
23.3 Known Assumptions	46
23.4 Known Gaps (Tracked Deliberately)	46
24. Future Roadmap	47
24.1 Short Term	47
24.2 Medium Term	47
24.3 Long Term	47
25. Testing & Quality Assurance	48
25.1 Running Tests.....	48
25.2 Component-Level Verification (No Network Calls).....	48
25.3 Coverage Expectations	48
25.4 A Working Principle: Isolate Before Wiring	49
26. Troubleshooting Guide	50
27. Appendix A — Facts, Assumptions & Recommendations	52

A.1 Facts.....	52
A.2 Assumptions	52
A.3 Recommendations.....	52
28. Appendix B — Glossary of Terms.....	53
29. Appendix C — Operational Command Reference	54
C.1 Training the Model	54
C.2 Exporting Synthetic Data	54
C.3 Benchmarking the Model	54
C.4 Running Every Test File Separately.....	54
30. Document Summary	55

1. Executive Summary

A compact, explainable Dubai mobility-intelligence system that turns live traffic, weather, and route data into a single drive-versus-metro recommendation.

Masar (مسار — “path” or “route”) is a Dubai-focused traffic and multimodal commute decision system built around a LangGraph workflow. The system predicts whether current conditions at a location are anomalously congested, escalates low-confidence predictions through a context-enrichment reasoning step, recommends a travel mode (drive, drive-to-metro, or metro), and returns the guidance in both English and Arabic.

The project serves two purposes simultaneously: it addresses a genuine Dubai commuting problem — road congestion, not transit delay, is typically the deciding factor in mode choice — and it functions as a flagship, interview-ready demonstration of applied AI engineering: multi-agent orchestration, tabular ML, graph-based routing, and live data integration working together in one deployed system.

The current build is best understood as a working local prototype extended into a deployed demo, not a finished production transportation platform. It is fully testable without any network calls (fake LLMs and synthetic data are supported throughout), while also supporting live TomTom, OpenWeatherMap, Supabase, and Groq integration when credentials are present.

1.1 Objectives

- Predict whether a route or location is experiencing anomalous congestion.
- Route low-confidence predictions through a deeper, LLM-assisted reasoning path.
- Recommend a specific travel mode and route under current conditions.
- Return bilingual English/Arabic guidance suited to a Dubai audience.
- Operate on local demo data or live Supabase/traffic-feed data interchangeably.
- Remain fully testable without external network calls, using fake LLMs and synthetic data.

1.2 Target Users & Stakeholders

Group	Role
Dubai commuters & mobility planners	Primary consumers of route/mode recommendations
Engineers validating the system	Verify AI-driven recommendations against real conditions
Demonstrators / product teams	Use the app as a working local mobility prototype
System owners / maintainers	Own the roadmap, data quality, and model lifecycle
Data & model providers	TomTom, OpenWeatherMap, Supabase, Groq

1.3 Success Criteria

The system is considered successful when it can, end to end:

1. Produce a valid congestion probability and confidence score for a given location.
2. Correctly execute both the fast path and the deep (context-enriched) path through the graph.
3. Produce bilingual recommendations from stateful, structured inputs.

4. Serve a demo route recommendation with zero network dependencies.
5. Train a model on synthetic or real traffic data and persist a usable artifact.

1.4 Scope Summary

Category	Coverage
In Scope	Dubai corridor modeling · local route recommendations · congestion scoring · synthetic training data · web demo and API surface · LLM-driven reasoning hooks
Out of Scope	Full turn-by-turn navigation · official RTA data integration · production authentication/RBAC · event-stream ingestion beyond the current collector · multi-tenant production deployment and observability

2. Project Overview & Scope

Masar frames Dubai commute planning as a structured, stateful AI workflow rather than a static map or a simple rule engine. Each recommendation request is modeled as a single pass through a LangGraph state machine: a location's live or manually supplied conditions become a typed state object, which is progressively enriched — predicted, routed, optionally reasoned about, optimized, and synthesized — into a final bilingual answer.

2.1 Core Feature Set

- Congestion prediction using a trained XGBoost classifier, with a heuristic fallback when no model artifact is present.
- Dynamic graph routing via LangGraph, including a fast path and a fallback deep-reasoning path.
- A context agent that explains low-confidence forecast scenarios using an LLM.
- NetworkX-based route optimization across a curated set of Dubai hubs and districts.
- Bilingual (English/Arabic) synthesis of the final recommendation.
- A local web app with a map view, manual inputs, and live traffic integration.
- Synthetic data generation so the model can be trained without production data.

2.2 Repository Root Structure

```
Masar/
├── Dockerfile
├── README.md
├── context.py
├── features.py
├── tomtom_route_sample.json
├── data/
├── masar/
│   ├── __init__.py
│   ├── live_traffic.py
│   └── nodes/
│       ├── __init__.py
│       ├── context.py
│       ├── optimizer.py
│       ├── predictor.py
│       ├── router.py
│       └── synthesis.py
├── models/
├── scripts/
│   ├── benchmark_model.py
│   ├── collect_traffic.py
│   ├── test_groq.py
│   ├── check_components.py
│   ├── export_synthetic.py
│   ├── train_model.py
│   ├── check_env.py
│   ├── collect_chatter.py
│   └── test_database_insert.py
├── sql/supabase_reddit_chatter.sql
├── tests/ (11 test modules – see Section 23)
├── VE/ (local virtual environment)
├── docker-compose.yaml
├── pytest.ini
├── requirements.txt
├── LICENSE
├── router.py
├── predictor.py
├── optimizer.py
├── synthesis.py (legacy/root nodes)
├── graph.py
├── state.py
├── map.py
├── train_model.py
├── tester.py
├── web_app.py
├── docs/archive/
└── frontend/
```

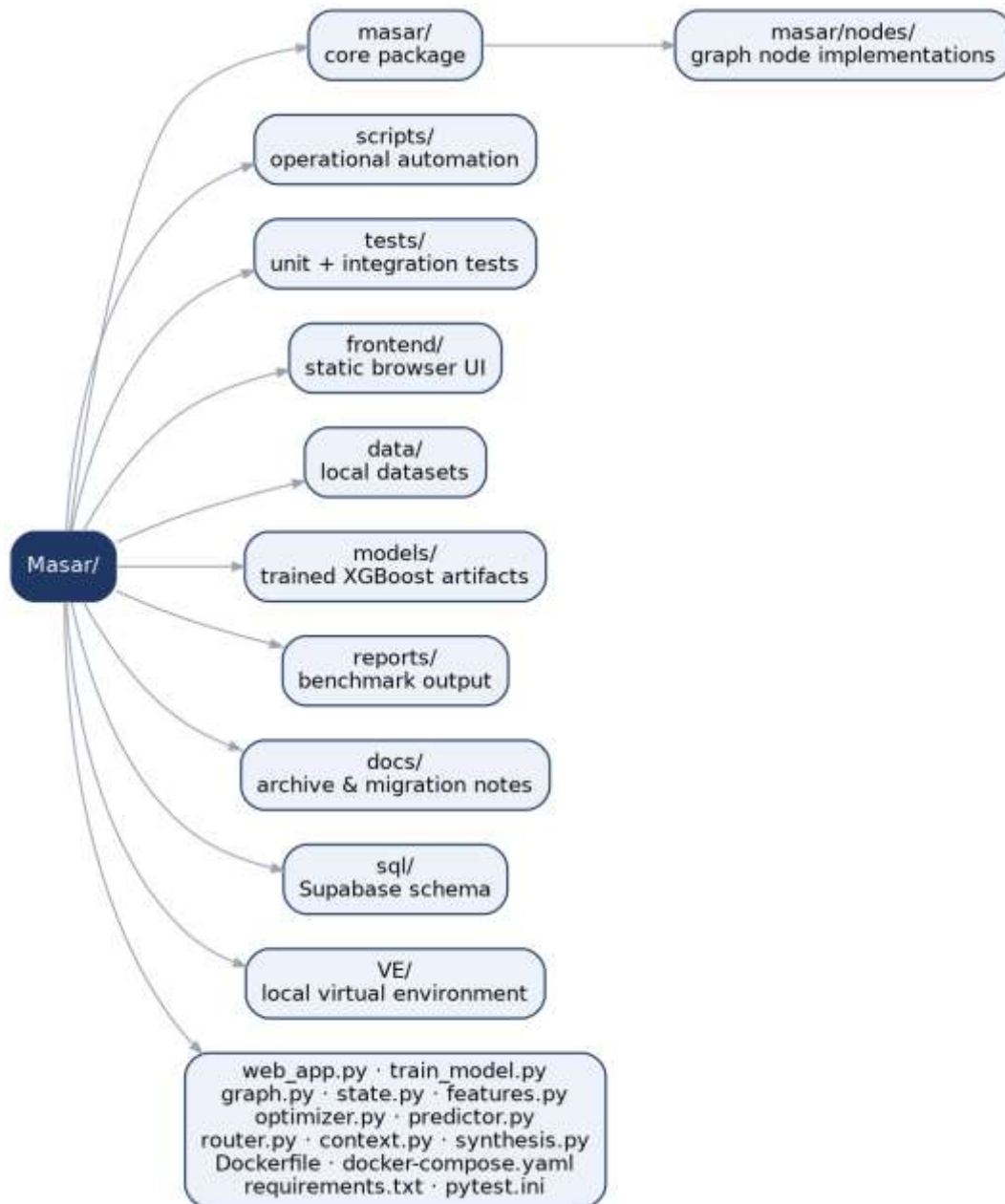


Figure 2.1 — Simplified workspace directory tree.

2.3 Folder Responsibilities

Folder	Purpose	Interacts With
masar/	Core application package: model, graph, state, live data, routing logic	Imported by the app and training scripts
masar/nodes/	Workflow node implementations (predictor, router, context, optimizer, synthesis)	Executed by masar.graph.build_graph()
scripts/	Operational automation: data collection, env checks, benchmarking, training, export	Invoked manually or by CI
tests/	Quality gate — unit and integration tests per component	Run via pytest
frontend/	Static browser UI (HTML/CSS/JS)	Served by web_app.py
data/	Local datasets — raw or derived snapshots	Used by scripts and analysis

Folder	Purpose	Interacts With
models/	Trained XGBoost model artifacts and local checkpoints	Loaded by masar/nodes/predictor.py
reports/	Generated benchmark reports	Used for model evaluation and documentation
docs/	Historical/archival reference material	Migration context only
sql/	Database schema definitions for Supabase	Supports traffic and chatter setups
VE/	Local virtual environment	Local development only

3. Business Problem & Solution

3.1 The Problem

A Dubai commuter needs actionable, real-time guidance that accounts for traffic conditions, route-level congestion, weather, key corridors, and likely delay — but road congestion, not transit reliability, is usually the dominant factor in whether driving or taking the metro is the better choice. Static maps show current traffic; they do not reason about mode choice under changing conditions, nor do they explain unusual congestion or provide guidance in the commuter's preferred language.

3.2 The Solution

Masar models this decision explicitly as a multi-step AI workflow: a fast, cheap prediction path handles the common case, while a slower, LLM-assisted path only activates when the model itself is uncertain. This mirrors how a human dispatcher would operate — trust the numbers when they are clear, dig for context when they are not — and keeps the system both responsive and explainable.

NOTE

Road congestion — not mass-transit delay — is the variable this system optimizes for. This is a deliberate, Dubai-specific modeling choice: metro reliability in Dubai is comparatively high, so the deciding factor in mode choice is almost always how bad the roads are.

4. Functional & Non-Functional Requirements

4.1 Functional Requirements

ID	Requirement	Source
FR-1	System shall accept origin, destination, and current traffic/weather inputs and return a congestion prediction with a confidence score.	predictor_node()
FR-2	System shall route low-confidence predictions to a context-enrichment step before route optimization.	router_node()
FR-3	System shall recommend one of drive / drive_to_metro / metro based on predicted congestion.	route_optimizer_node()
FR-4	System shall return the final recommendation in both English and Arabic.	synthesis_node()
FR-5	System shall optionally refresh state from live Supabase traffic rows before running the graph.	web_app.py, live_traffic.py
FR-6	System shall expose a health endpoint and a recommendation endpoint over HTTP.	web_app.py
FR-7	System shall support training a congestion classifier from synthetic and/or real data.	train_model.py
FR-8	System shall run entirely offline/demo (fake LLM, synthetic data) for local testing.	tests/, demo mode

4.2 Non-Functional Requirements

Category	Requirement
Explainability	Model and routing decisions must remain simple enough to reason about and debug — an interpretable tabular model and a curated corridor graph rather than opaque deep models.
Testability	Every node must be independently testable in isolation, without live network calls (see Section 23).
Portability	Frontend and API must be served from a single process/container to simplify deployment (see Section 18).
Resilience	Missing model artifacts, missing credentials, or provider failures must degrade gracefully rather than crash the request.
Reproducibility	Model training must be seed-controlled and use a chronological split to avoid temporal leakage.
Bilingual UX	Every user-facing recommendation must be produced in English and Arabic.
Observability (baseline)	External API calls should be logged in redacted form for later inspection (see Section 20).

5. High-Level System Architecture

Masar is architected as a single Python process (`web_app.py`) that serves both the static frontend and the JSON API, wrapping a compiled LangGraph state machine. A request arrives as JSON, is converted into a typed `MasarState`, optionally enriched with a live Supabase/TomTom/OpenWeatherMap row, and then passed through the graph. The graph always runs the predictor and router; it conditionally runs the context agent depending on prediction confidence; it always runs the route optimizer and synthesis nodes before returning.

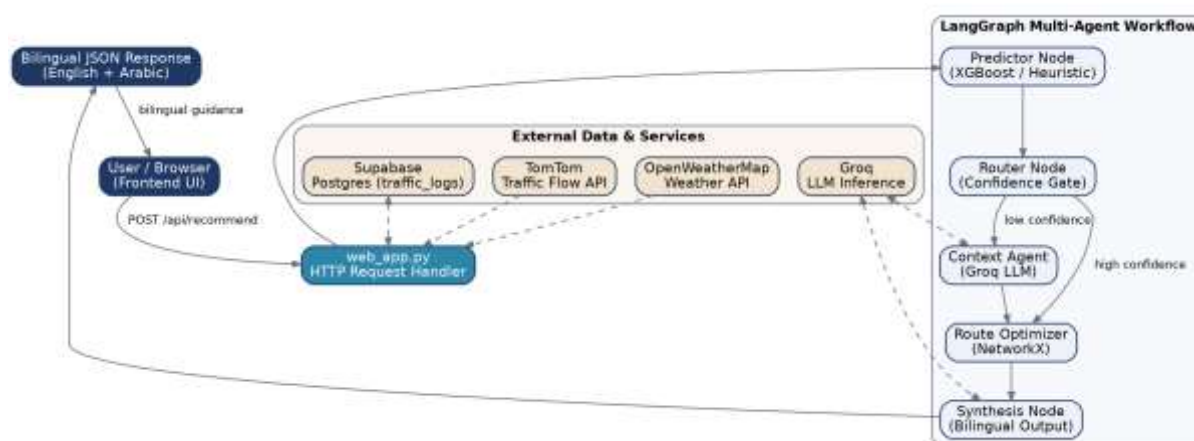
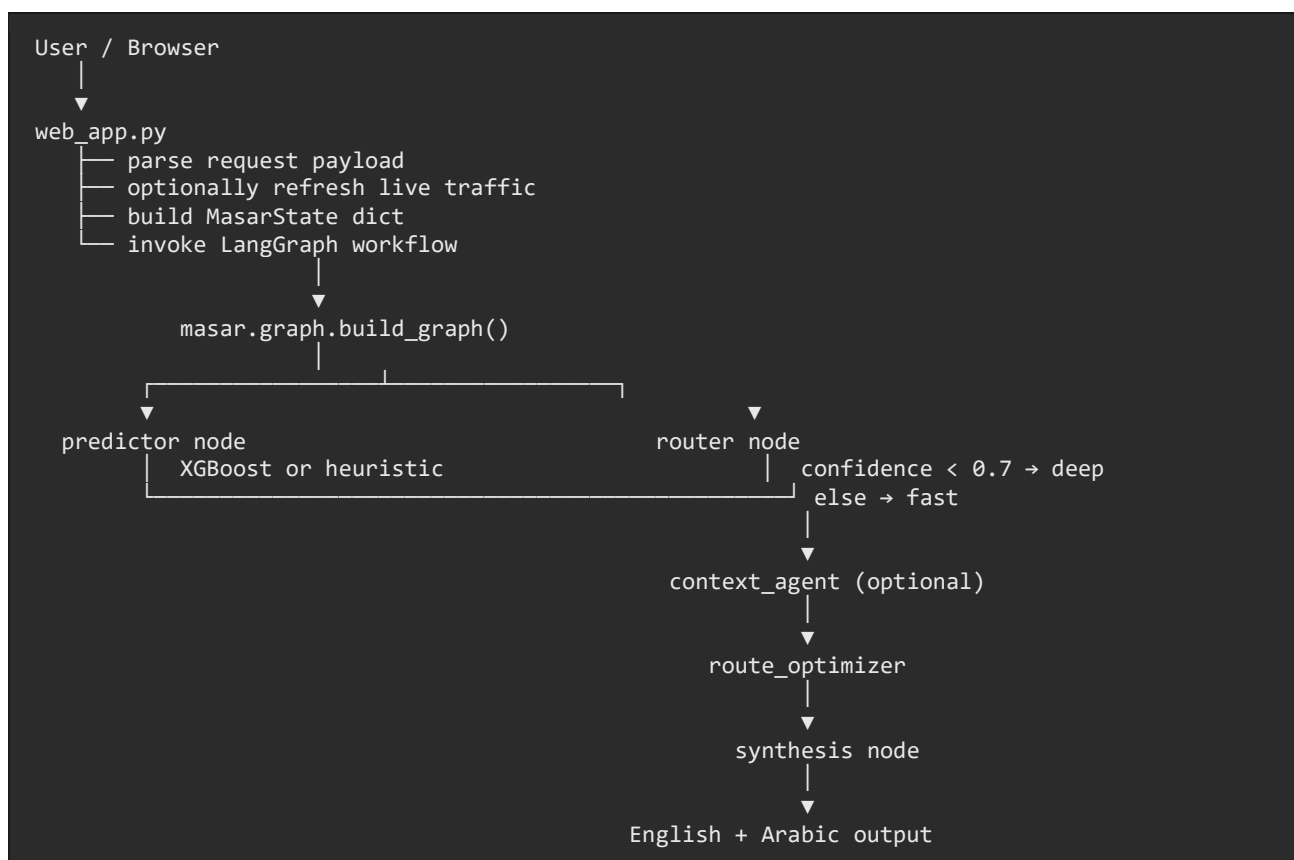


Figure 5.1 — High-level system architecture: client, application layer, LangGraph workflow, and external services.

5.1 Runtime Flow (Textual)



6. Software Architecture (Layered View)

The codebase separates cleanly into six logical layers. No layer reaches more than one level down except where explicitly noted (e.g., the orchestration layer calling directly into the model layer for inference).

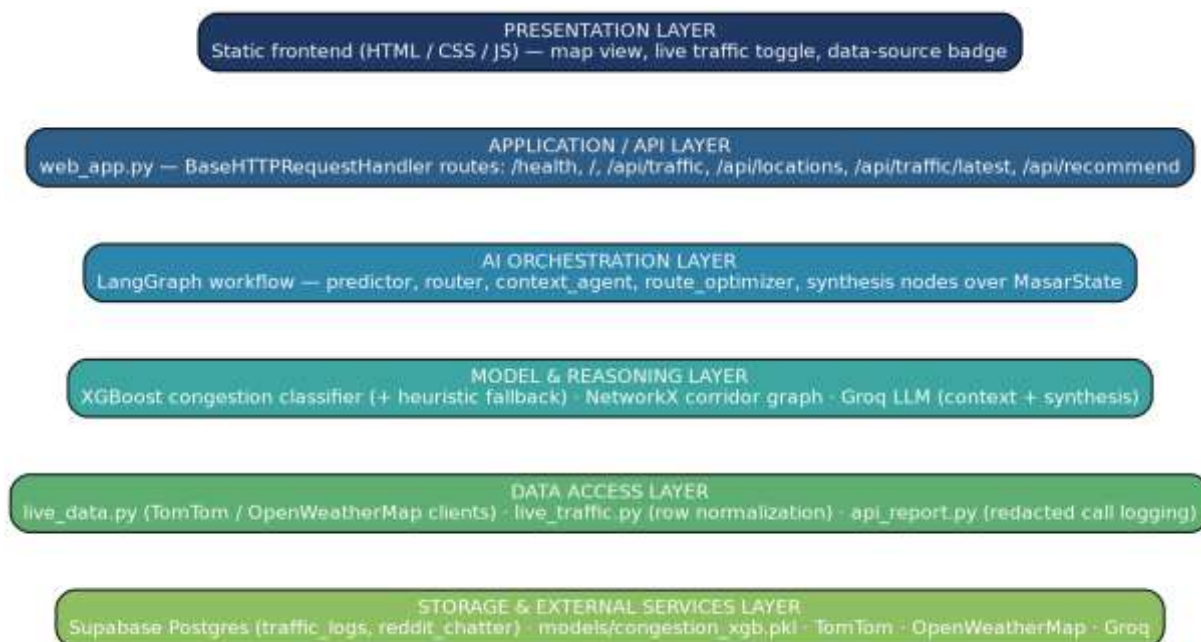


Figure 6.1 — Layered architecture, presentation through storage.

Layer	Responsibility	Key Modules
Presentation	Renders the map, live-traffic toggle, and data-source badge in the browser	frontend/index.html and related assets
Application / API	Parses HTTP requests, assembles state, returns JSON	web_app.py
AI Orchestration	Executes the LangGraph workflow over MasarState	masar/graph.py, masar/nodes/*
Model & Reasoning	Produces predictions, routes, and language output	XGBoost model, NetworkX graph, Groq LLM
Data Access	Normalizes and fetches external data	live_data.py, live_traffic.py, api_report.py
Storage & External Services	Persists and supplies raw signal	Supabase Postgres, TomTom, OpenWeatherMap, model artifacts

7. Workspace & Directory Structure — Detail

Section 2.2 shows the full repository tree; this section documents the lifecycle each top-level folder participates in, from source to runtime.

Path	Lifecycle Stage	Notes
masar/, masar/nodes/	Import time → request time	Loaded once by web_app.py / graph.py; node functions execute per request
scripts/	Build time / scheduled time	Run manually or via GitHub Actions (collect_traffic.py, train_model.py)
tests/	CI / pre-deploy time	Executed via pytest before merges or releases
models/	Build time → request time	congestion_xgb.pkl is baked at Docker build time and loaded at inference time
data/, reports/	Analysis time	Synthetic exports and benchmark reports for offline review
frontend/	Serve time	Static assets returned directly by web_app.py

8. Complete File-by-File Documentation

Every entry below reflects a file that exists in the repository snapshot. No files, functions, or APIs beyond what the project documents are represented here.

File	Purpose	Imported By	Key Dependencies
web_app.py	Local recommendation HTTP server and demo app	Run directly	masar.graph, masar.live_traffic, masar.locations
masar/graph.py	Compiles the LangGraph workflow	web_app.py, tests	All node modules, MasarState
masar/state.py	Shared state contract (TypedDict)	All nodes and graph	typing, operator
train_model.py	Trains the anomaly classifier	CLI entrypoint	masar.features, masar.locations, xgboost
masar/nodes/predictor.py	Congestion prediction	graph	joblib, numpy, masar.features
masar/nodes/router.py	Chooses deep vs. fast path	graph	langgraph.types.Command
masar/nodes/context.py	Context enrichment and reasoning	graph	langchain_groq, masar.api_report
masar/nodes/optimizer.py	Route decision and mode selection	graph	networkx, masar.locations
masar/nodes/synthesis.py	Bilingual final output	graph	pydantic, LLM output
masar/live_data.py	Live TomTom / OpenWeather fetching	masar/live_traffic.py	httpx, masar.api_report
masar/live_traffic.py	Maps live data to state fields	web_app.py	masar.locations
masar/locations.py	Canonical Dubai location catalog	optimizer, live_data, web_app	static list
masar/features.py	Feature transforms / FEATURE_ORDER	train_model.py, predictor	numpy
masar/api_report.py	Redacted API call reporting	all external call sites	json, os, time
scripts/collect_traffic.py	Background traffic collection	CI or manual run	httpx, supabase, dotenv
scripts/collect_chatter.py	Social signal collection (stub/paused)	manual run	praw, dotenv
scripts/check_components.py	End-to-end component checks	manual run	local modules
map.py	TomTom route comparison helper	ad hoc use	httpx, local JSON
tester.py	Environment and service smoke tests	manual run	external environment

9. Technology Stack & Dependency Report

9.1 Runtime Environment

Item	Value	Notes
Python version	3.11	Confirmed by Dockerfile: FROM python:3.11-slim
Virtual environment	Local VE/ folder	Used for local developer environment
Package manager	pip via requirements.txt	No pyproject.toml in the current snapshot
Operating systems	Windows (local env) · Linux (Docker container)	Dockerfile targets Linux

9.2 Dependency Inventory

Package	Why It Is Used
langgraph	State-machine orchestration and conditional graph routing
langchain-groq	Groq LLM client integration
langchain-core	Core LLM abstractions and message types
pydantic	Structured output validation for the synthesis node
joblib	Model serialization and deserialization
numpy	Numeric array generation and feature engineering
pandas	Data processing and training-frame assembly
openpyxl	Excel export support for synthetic data
scikit-learn	Metrics and model-pipeline utilities
xgboost	Primary congestion anomaly classifier
networkx	Route-graph construction and shortest-path logic
pytest	Automated testing
python-dotenv	Local environment variable loading
supabase	Traffic data access from Supabase
httpx	Async HTTP client for TomTom and OpenWeatherMap
praw	Reddit chatter collection support (integration paused)

Versions are currently unpinned in requirements.txt. Freezing exact versions is tracked as a short-term roadmap item (Section 25).

9.3 Runtime Requirements

Requirement	Value	Notes
CPU	Standard x86_64	XGBoost runs on CPU by default
GPU	Optional, not required	No CUDA/cuDNN configuration is declared

Requirement	Value	Notes
RAM	~2–8 GB typical	Sufficient for model and request processing
Storage	Low	Mostly source code plus a small number of model artifacts
Network	Required for live traffic, Groq, and Supabase	Demo mode avoids external dependencies entirely

9.4 Environment Variables

Variable	Example / Default	Required	Purpose
SUPABASE_URL	https://...supabase.co	For live data / real training	Supabase traffic database
SUPABASE_KEY	secret key	For live data / real training	Supabase authentication
GROQ_API_KEY	secret key	For live LLM reasoning	LLM access
GROQ_MODEL	openai/gpt-oss-120b	No	Overrides the default LLM model
TOMTOM_API_KEY	secret key	For live traffic	TomTom flow API
OPENWEATHER_API_KEY	secret key	For live weather	OpenWeatherMap API
MASAR_MODEL_PATH	models/congestion_xgb.pkl	No	Overrides the model path
MASAR_API_REPORT_PATH	file path	No	Persists redacted API call data
PORT / MASAR_PORT	8000	No	HTTP server port
MASAR_HOST	127.0.0.1 or 0.0.0.0	No	Bind address — must be 0.0.0.0 in containers

9.5 Secrets & Feature Flags

- Secrets are expected in .env or process environment variables and are never committed to source control.
- demo mode disables real Groq calls for the local recommendation workflow.
- use_live_traffic toggles live provider calls on or off; missing credentials fall back to manual input.
- MASAR_MODEL_PATH allows a non-default trained-model path.
- MASAR_API_REPORT_PATH enables redacted API/model call logging for later inspection.

10. AI Agents & Model Architecture

Masar's reasoning layer is organized as five cooperating agents/nodes inside one LangGraph workflow. Each has a single responsibility, a defined input/output contract over MasarState, and an explicit failure mode.

Agent	Responsibility	Goal	Tooling
Predictor	Estimate congestion	Output a predicted score + confidence	XGBoost model or heuristic fallback
Router	Decide reasoning path	Send low-confidence cases to the context route	LangGraph Command
Context Agent	Diagnose unusual congestion	Explain the likely cause using available signals	Groq LLM or injected fake LLM (tests)
Route Optimizer	Find route and mode	Compute the best route under current congestion	NetworkX Dijkstra shortest path
Synthesis Agent	Produce the final response	Return a short bilingual recommendation	Pydantic-validated Groq LLM output

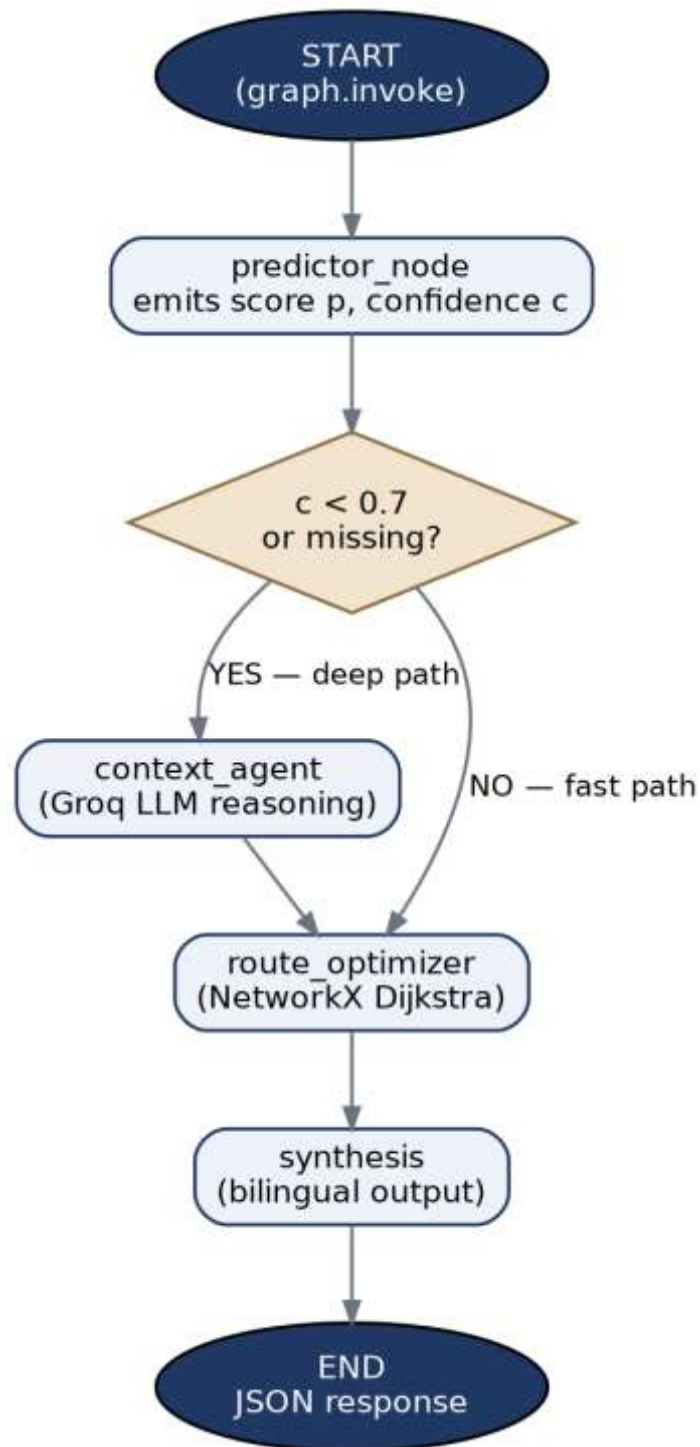


Figure 10.1 — Predictor → Router decision gate → fast/deep path → Synthesis.

10.1 Failure Behavior by Agent

Condition	Behavior
Missing model artifact	Predictor falls back to heuristic scoring with a fixed low confidence (0.4)
Missing / undefined confidence	Router treats the case as anomalous and takes the deep path
Missing provider credentials	Live traffic lookup fails quietly or reverts to manual input
No route path available	Optimizer returns empty route data but still offers a mode recommendation

Condition	Behavior
LLM failure	Demo mode avoids network dependency; outside demo mode, errors may surface in the API response

10.2 Congestion Classifier (XGBoost)

Item	Value
Model name	XGBClassifier
Artifact path	models/congestion_xgb.pkl
Provider	Local training pipeline (train_model.py)
Parameters	n_estimators=200, max_depth=4, learning_rate=0.05, scale_pos_weight balancing
Context window / tokens / embeddings	Not applicable — tabular model
Quantization	Not implemented

Why chosen: the project is structured as a small, explainable anomaly detector for traffic behavior rather than a deep neural network. XGBoost suits tabular location/time/weather data and supports low-overhead local inference — an appropriate match for an interpretable, easily debugged system.

KNOWN GAP

As of the current build, models/congestion_xgb.pkl does not yet exist in the repository. predictor.py therefore permanently uses the heuristic fallback (fixed confidence 0.4). Completing the training pipeline is the top short-term priority — see Section 24.

10.3 LLM (Groq)

Item	Value
Default model	openai/gpt-oss-120b
Provider	Groq
Temperature	0.2 (context node) · 0.3 (synthesis node)
Max output	Not explicitly limited in code
Streaming	Not implemented in the current graph
Prompting pattern	Brief, state-driven prompts with route and weather context

- The context node passes a short, factual prompt describing location, congestion ratio, and weather.
- The synthesis node is instructed to write a short English message followed by the same message in Arabic.
- Structured outputs are validated via pydantic.BaseModel (SynthesisOutput).

11. Data Pipeline — Live Traffic & Weather Ingestion

Masar consumes live signal from TomTom (flow speed), OpenWeatherMap (weather/rain), and Supabase (persisted traffic_logs rows), alongside locally generated synthetic rows used for offline model training.

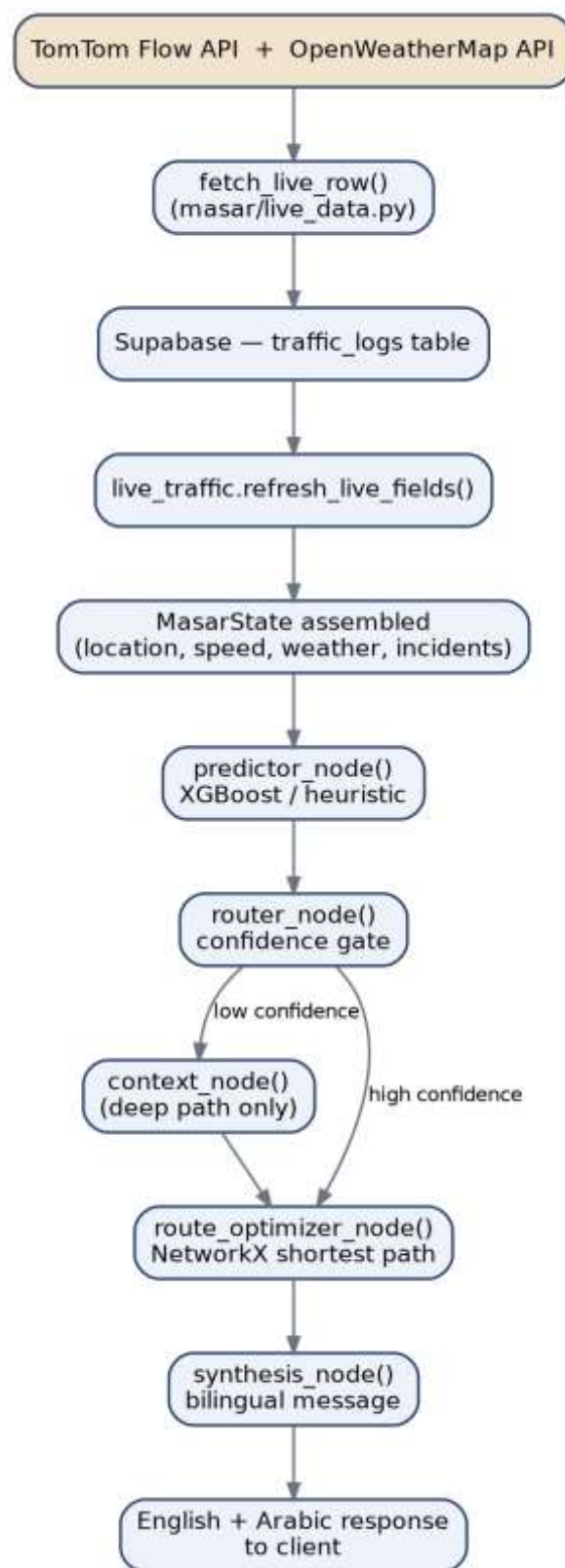


Figure 11.1 — Traffic and weather ingestion into the recommendation graph.

11.1 Source Tables

Table / Dataset	Source	Key Fields	Notes
traffic_logs	Supabase	location_name, current_speed, free_flow_speed, speed_ratio, weather_main, rain_mm, raw_data, created_at	Main runtime signal
reddit_chatter	Optional external feed	reddit_id, title, score, post_url, published_at	Not actively wired into the core graph
Synthetic training rows	train_model.py	Location-related features + anomaly labels	Used for local model validation

11.2 Feature Schema for Training

Field	Type	Description
current_speed	float	Observed speed
free_flow_speed	float	Expected uncongested speed
congestion_ratio	float	current_speed / free_flow_speed
temperature	float	Weather temperature (defaults to 30.0 if missing)
is_raining	bool	Whether rain is present (rare, high-impact binary flag)
hour	int	Traffic hour bucket
is_weekend	bool	Friday–Saturday UAE weekend indicator
location	str	District or corridor label
timestamp	datetime	Row timestamp
source	str	synthetic or real

11.3 Data Cleaning & Validation

- Values are normalized from provider JSON into consistent keys.
- Missing weather temperature values fall back to a default of 30.0.
- Synthetic rows are explicitly marked source="synthetic".
- Real rows are mapped into the same feature schema before training.

11.4 Train / Validation / Test Split

The split is time-based rather than random: `split_idx = int(len(df) * 0.8)`, where earlier rows form the training set and later rows form the held-out test set. This avoids temporal leakage and better simulates real-world deployment, where the model must generalize to traffic patterns it has not seen yet.

12. LangGraph Workflow & Component-Level Design

This section documents each node module's contract — inputs, outputs, dependencies, and complexity — followed by the static component and data-model views of the package.

`masar/nodes/predictor.py`

Field	Detail
Purpose	Predict whether a location/time pattern is anomalous and estimate a confidence value
Input	MasarState (speed, free-flow speed, congestion ratio, weather, rain, temperature)
Output	predicted_congestion, prediction_confidence
Dependencies	joblib, numpy, masar.features, masar.state
Error Handling	Falls back to a heuristic model if the artifact is missing
Complexity	O(1) per prediction (dominated by inference)

`masar/nodes/router.py`

Field	Detail
Purpose	Choose between fast and deep reasoning paths
Input	prediction_confidence and related state
Output	Command with update + route destination
Dependencies	langgraph.types.Command, masar.nodes.predictor
Error Handling	Missing confidence safely triggers the deep path
Complexity	O(1)

`masar/nodes/context.py`

Field	Detail
Purpose	Add human-readable context for low-confidence conditions
Input	State + optional LLM client
Output	context_notes, nearby_events, social_signal
Dependencies	langchain_groq or injected LLM stub
Error Handling	Graceful fallback to stub text when integrations are absent
Complexity	O(1) + LLM latency

`masar/nodes/optimizer.py`

Field	Detail
Purpose	Choose a route and mode via weighted graph shortest-path logic
Input	Predicted congestion, origin, destination, route state

Field	Detail
Output	recommended_mode, recommended_route, route_options
Dependencies	networkx, masar.locations
Error Handling	Returns empty route data if no path is available
Complexity	$\sim O(E \log V)$

masar/nodes/synthesis.py

Field	Detail
Purpose	Convert route guidance into bilingual output
Input	Final route state and context notes
Output	message_en, message_ar
Dependencies	pydantic, LLM invocation
Error Handling	Validates structured output via Pydantic
Complexity	$O(1)$ + LLM latency

masar/live_data.py

Field	Detail
Purpose	Fetch real-time TomTom and weather data into a shaped traffic row
Input	Location label + configured API keys
Output	Row dict (speeds, weather, raw provider values)
Dependencies	httpx, asyncio, masar.locations
Error Handling	Raises on missing credentials or provider failure
Complexity	$O(1)$ network calls per location

masar/live_traffic.py

Field	Detail
Purpose	Read and normalize the latest traffic row into state fields
Input	Supabase client + location label
Output	fields, row, age_seconds
Dependencies	masar.locations, masar.live_data
Error Handling	Returns empty fields when no row is available
Complexity	$O(1)$

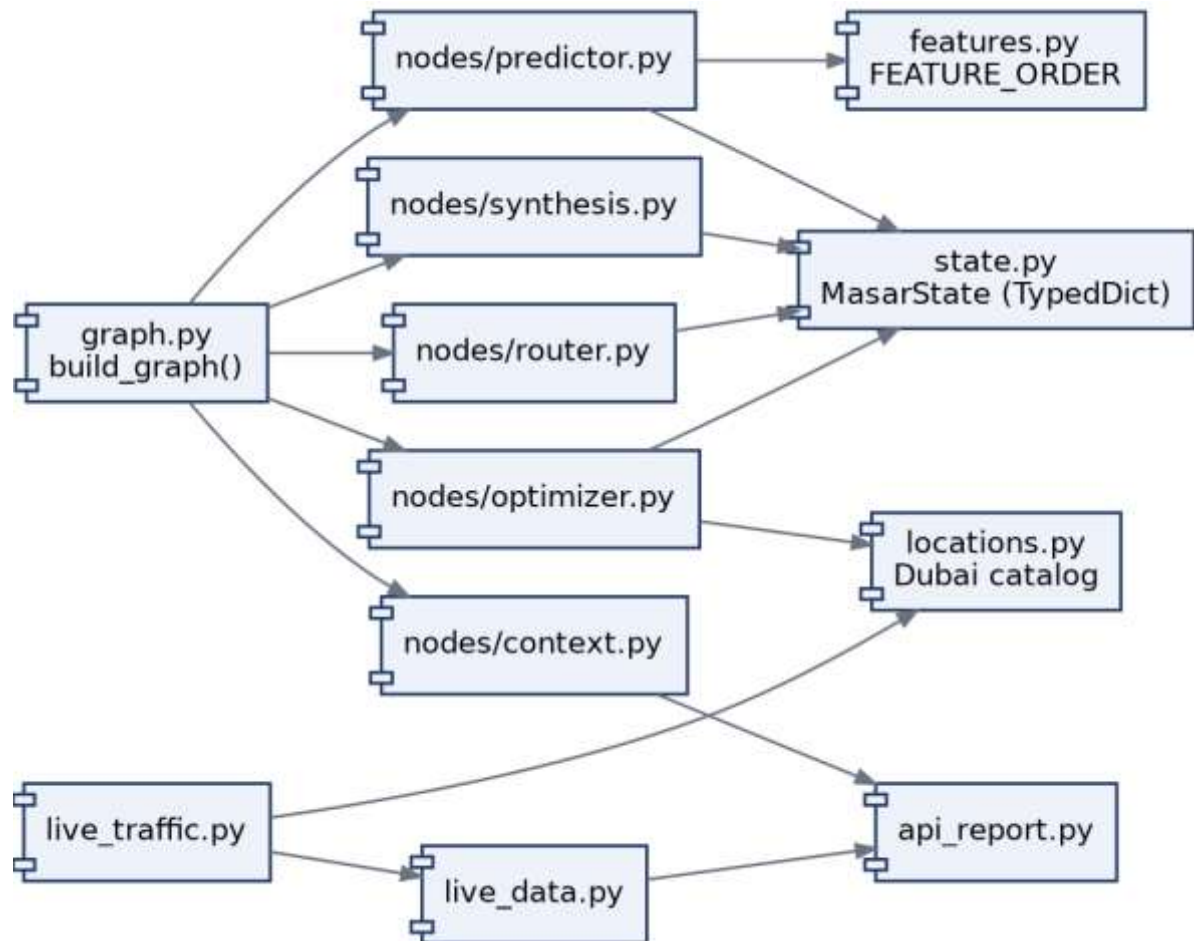


Figure 12.1 — Component dependency graph across the masar package.

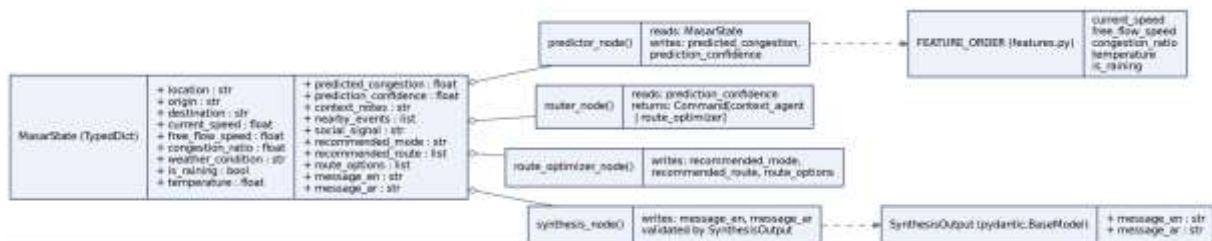


Figure 12.2 — MasarState data contract and the node functions that read/write it.

13. Model Training & MLOps Pipeline

Feature engineering is deliberately compact and centralized in a single source of truth so that the predictor and the trainer can never drift out of sync.

```
FEATURE_ORDER = [
    "current_speed",
    "free_flow_speed",
    "congestion_ratio",
    "temperature",
    "is_raining",
]
```

Feature categories: numerical (speed, free-flow speed, temperature, congestion ratio) · binary (is_raining) · location-based categorical labels (e.g., Marina, Business Bay) · contextual aggregates (time-of-day bucket, weekend flag) generated during training.

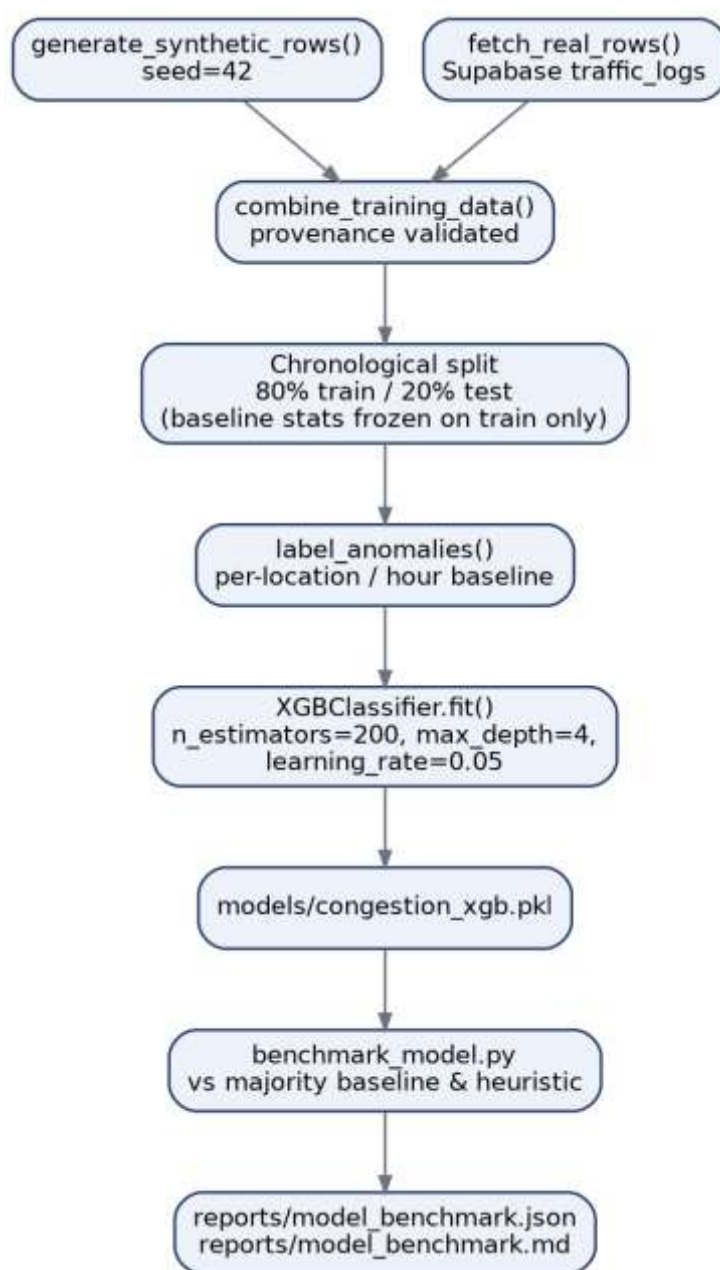


Figure 13.1 — Training data lifecycle from synthetic/real rows to a persisted, benchmarked model.

13.1 Reproducibility & Versioning

- seed=42 is used by default in synthetic generation.
- The time-based train/test split reduces temporal leakage.
- The model artifact is persisted to models/congestion_xgb.pkl.
- There is no formal model registry or MLflow integration in the current snapshot — the project is artifact-based rather than registry-based.

13.2 Training Artifacts

Artifact	Description
models/congestion_xgb.pkl	Serialized trained model
reports/model_benchmark.json	Machine-readable benchmark output
reports/model_benchmark.md	Human-readable benchmark report
tomtom_route_sample.json	Sample route payload

13.3 Current Benchmark (Synthetic Data)

6,000 synthetic rows were used — 4,800 for training, 1,200 held out for testing, with 90 anomalous rows in the test split. Because labels are generated by the synthetic z-score rule, these metrics measure how faithfully the model reproduces that rule, not real-world traffic accuracy.

Benchmark	Accuracy	Balanced Acc.	Precision	Recall	F1	ROC-AUC	PR-AUC
Majority baseline	0.925	0.500	0.000	0.000	0.000	0.500	0.075
Heuristic (1 – congestion_ratio)	0.807	0.656	0.189	0.478	0.270	0.858	0.386
XGBoost	0.928	0.925	0.509	0.922	0.656	0.982	0.871

KNOWN GAP

Real-only metrics must be reported once independently labeled traffic outcomes are available. As of this build, real Supabase coverage is roughly 81 rows (~1.5 days) — not enough for a meaningful per-location/hour baseline. A realistic target before retraining on real data alone is 2–3 weeks of coverage.

RISK

train_model.py currently has a label-leakage issue:

Benchmark	Accuracy	Balanced Acc.	Precision	Recall	F1	ROC-AUC	PR-AUC
baseline statistics are computed from the full dataset before splitting. The corrected implementation must split chronologically first and freeze baseline statistics from the training partition only.							

14. Function-Level API Documentation

Every function documented below exists in the current source tree. Signatures, parameters, return types, and complexity are drawn directly from the project's own inline documentation.

14.1 train_model.py

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>generate_synthetic_rows(n_rows, seed=42)</code>	Generate synthetic Dubai congestion data for local training/validation	<code>n_rows: int</code> , <code>seed: int</code>	<code>pd.DataFrame</code>	Called by <code>main()</code>	$O(n_rows)$
<code>fetch_real_rows(client=None)</code>	Load traffic rows from Supabase without writing data	<code>client: optional</code>	<code>pd.DataFrame</code>	Called by <code>main()</code> ; calls <code>measured_call()</code> , <code>row_to_features()</code>	$O(n)$
<code>combine_training_data(*frames)</code>	Concatenate training frames while validating provenance	<code>frames: pd.DataFrame.</code> ..	<code>pd.DataFrame</code>	Called by <code>main()</code> ; raises <code>ValueError</code> if source $\notin$ {real, synthetic}	$O(n)$
<code>label_anomalies(df)</code>	Compute a per-location time-of-day baseline and label anomalies	<code>df: pd.DataFrame</code>	<code>pd.DataFrame</code> with <code>is_anomaly</code> column	Called by <code>main()</code>	$O(n)$
<code>train(df)</code>	Train a gradient-boosted anomaly classifier	<code>df: pd.DataFrame</code>	<code>XGBClassifier</code>	Called by <code>main()</code> ; calls <code>model.fit()</code>	$O(n \times \text{trees})$
<code>main()</code>	CLI entrypoint for training and saving the model artifact	CLI args	None	Direct execution; raises <code>ValueError</code> if dataset lacks both classes	$O(n \times \text{trees})$

14.2 masar/nodes/predictor.py

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>_load_model()</code>	Cache a model object from disk if available	—	model or None	Called by <code>predictor_node()</code> ; calls <code>joblib.load()</code>	$O(1)$ after first load
<code>_build_feature_vector(state)</code>	Build a 1×N feature vector matching training columns	<code>state: MasarState</code>	<code>np.ndarray</code>	Called by <code>predictor_node()</code>	$O(F)$

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>_heuristic_predict(state)</code>	Fallback rule-based congestion estimate when no model exists	state: MasarState	(float, float) score + low confidence	Called by <code>predictor_node()</code>	O(1)
<code>predictor_node(state)</code>	Top-level prediction node attached to the graph	state: MasarState	dict: <code>predicted_congestion</code> , <code>prediction_confidence</code>	Called by graph runtime	O(1) or model inference

14.3 masar/nodes/router.py

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>router_node(state)</code>	Determine whether the request routes to context or optimizer	state: MasarState	Command["context_agent" "route_optimizer"]	Called by graph runtime	O(1)

14.4 masar/nodes/context.py

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>_get_social_signal(location)</code>	Return a placeholder social signal (Reddit integration inactive)	location: str	str	Called by <code>context_node()</code>	O(1)
<code>_get_nearby_events(location)</code>	Return an empty list (no events service wired)	location: str	list[str]	Called by <code>context_node()</code>	O(1)
<code>_build_prompt(state, social_signal, events)</code>	Format the prompt for a low-confidence context explanation	state, social_signal, events	str	Called by <code>context_node()</code>	O(1)
<code>context_node(state, llm=None)</code>	Query an LLM to explain unusual congestion and enrich state	state: MasarState, llm: optional	dict: context notes + social/event metadata	Called by graph runtime; calls <code>measured_call()</code> , <code>llm.invoke()</code>	O(1) + LLM latency

14.5 masar/nodes/optimizer.py

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>_build_base_graph()</code>	Create a minimal corridor graph linking	—	networkx.Graph	Called by <code>route_optimizer_node()</code>	O(E)

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
	key Dubai hubs				
<code>_build_location_graph()</code>	Expand the graph with additional districts by nearest-hub approximation	—	<code>networkx.Graph</code>	Called by <code>route_optimizer_node()</code>	$O(V \log V)$
<code>_distance_minutes(source, target)</code>	Approximate geographic distance between two coordinates	<code>source, target: str</code>	<code>float</code>	Called by <code>_build_location_graph()</code>	$O(1)$
<code>_recommend_mode(congestion)</code>	Turn predicted congestion into drive / drive_to_metro / metro	<code>congestion: float</code>	<code>str</code>	Called by <code>route_optimizer_node()</code>	$O(1)$
<code>route_optimizer_node(state)</code>	Compute the best route and route options under congestion stress	<code>state: MasarState</code>	dict: <code>recommended_mode, recommended_route, route_options</code>	Calls <code>networkx.dijkstra_path()</code> ; handles <code>NodeNotFound</code> / <code>no-path</code>	$O(E \log V)$

14.6 masar/nodes/synthesis.py

Function	Purpose	Parameters	Returns	Called By / Calls	Complexity
<code>SynthesisOutput(model)</code>	Structured output schema for English + Arabic messages	—	Pydantic schema (required strings)	Used by <code>synthesis_node()</code>	—
<code>_build_prompt(state)</code>	Generate a short message prompt for bilingual synthesis	<code>state: MasarState</code>	<code>str</code>	Called by <code>synthesis_node()</code>	$O(1)$
<code>synthesis_node(state, llm=None)</code>	Produce the final bilingual output	<code>state, llm: optional</code>	dict: <code>message_en, message_ar</code>	Calls <code>llm.invoke()</code> / Groq structured output; Pydantic-validated	$O(1)$ + LLM latency

14.7 web_app.py

Function	Purpose
<code>make_state(payload)</code>	Convert HTTP JSON into a valid <code>MasarState</code>
<code>_get_supabase_client()</code>	Build a Supabase client from environment variables
<code>run_recommendation(payload)</code>	Build the graph, optionally refresh live traffic, and return a result payload
<code>traffic_snapshot()</code>	Read recent traffic rows from Supabase for the UI dashboard
<code>latest_traffic_for_location(location)</code>	Return the most recent row for a particular location
<code>MasarHandler.do_GET()</code>	Serve health, root page, traffic, and latest-traffic endpoints
<code>MasarHandler.do_POST()</code>	Accept POST <code>/api/recommend</code> and run the recommendation workflow

14.8 masar/live_data.py

Function	Purpose
<code>_fetch_tomtom(client, lat, lon, api_key)</code>	Fetch and normalize TomTom flow data
<code>_fetch_weather(client, lat, lon, api_key)</code>	Fetch and normalize OpenWeatherMap data
<code>fetch_live_row(location)</code>	Retrieve a fully assembled live traffic row from provider APIs
<code>fetch_and_store(location, supabase_client)</code>	Persist a row into Supabase after fetching live traffic

15. Backend API — Input / Output Specification

The project does not use FastAPI or Flask; it exposes a lightweight HTTP surface via Python's BaseHTTPRequestHandler inside web_app.py.

15.1 GET /health

Health check endpoint.

```
{"status": "ok"}
```

15.2 GET /

Serves the frontend page (HTML from frontend/index.html).

15.3 GET /api/traffic

Fetches recent traffic rows.

```
{
  "rows": [
    { "location_name": "...", "current_speed": 32.0, "free_flow_speed": 90.0 }
  ]
}
```

15.4 GET /api/locations

Lists map locations and coordinates.

```
[{"label": "Marina", "lat": 25.08, "lon": 55.14}]
```

15.5 GET /api/traffic/latest?location=Marina

Fetches the most recent data point for a specific location.

```
{"row": {"..."}, "fields": {"..."}, "age_seconds": 240.1}
```

15.6 POST /api/recommend

Runs a recommendation and returns the full graph output.

Request example:

```
{
  "demo": true,
  "use_live_traffic": false,
  "origin": "Marina",
  "destination": "Business Bay",
  "current_speed": 25,
  "free_flow_speed": 90,
  "temperature": 36,
  "weather_condition": "Clear",
  "is_raining": false
}
```

Response example:

```
{
  "result": {
    "predicted_congestion": 0.62,
    "prediction_confidence": 0.76,
    "recommended_mode": "drive_to_metro",
    "recommended_route": ["Marina", "Business Bay"],
    "message_en": "Use the recommended route and allow extra time for congestion.",
    "message_ar": "استخدم المسار الموصى به واترك وقتًا إضافيًا للازدحام."
  }
}
```

15.7 API Validation & Status Handling

- Request bodies are parsed as JSON.
- A fallback path re-runs the graph in demo mode if the regular mode fails.
- HTTP status handling uses 200, 400, 500, and 503 depending on the failure case.

16. Configuration & Environment Guide

16.1 Configuration Sources

- .env, loaded via `python-dotenv.load_dotenv()`
- OS environment variables
- Local code defaults — `MODEL_PATH`, `CONFIDENCE_THRESHOLD`, `GROQ_MODEL`
- JSON payloads on incoming HTTP requests
- Pydantic models for synthesis output validation

16.2 Variable Catalog

Variable	Description	Default	Required
<code>MODEL_PATH</code>	Path used by the predictor	<code>models/congestion_xgb.pkl</code>	No
<code>CONFIDENCE_THRESHOLD</code>	Router threshold for fast vs. deep path	0.7	No
<code>location</code>	Current place label used for routing	Provided at runtime	Yes
<code>origin</code>	Route origin	Marina	No
<code>destination</code>	Route destination	Business Bay	No
<code>congestion_ratio</code>	Current speed / free-flow speed	Derived	No
<code>weather_condition</code>	Weather label	Clear	No
<code>is_raining</code>	Rain flag	False	No
<code>demo</code>	Whether to fake LLM calls in the web app	True	No
<code>use_live_traffic</code>	Whether to fetch live data for a recommendation	True	No

16.3 Secrets Management

The project follows a typical local-development pattern: secrets live in .env and are read through `python-dotenv`. Secrets are not exposed in logs — `masar/api_report.py` redacts any key-like field name before persisting a record.

17. External Services & Integrations

Service	Purpose	Authentication	Status
TomTom Traffic API	Current speed and flow metadata	TOMTOM_API_KEY	Active in live_data.py
OpenWeatherMap	Weather and rainfall	OPENWEATHER_API_KEY	Active in live_data.py
Supabase	Persistence and traffic lookups	SUPABASE_URL + SUPABASE_KEY	Used for local data and dashboard
Groq	LLM reasoning	GROQ_API_KEY	Active in context and synthesis nodes
Reddit / PRAW	Social chatter collection	Credentials / app token	Stubbed / pending
GitHub Actions	Traffic collector automation	Repository secrets	Configured in workflows

17.1 TomTom

Endpoint pattern: <https://api.tomtom.com/.../flowSegmentData/absolute/12/json> — uses point, unit, and API key; returns current and free-flow speed plus time metrics.

17.2 OpenWeatherMap

Endpoint pattern: <https://api.openweathermap.org/data/2.5/weather> — uses lat, lon, units=metric, and API key; returns current weather, temperature, and rainfall.

17.3 Supabase

- Read-only selects are used in the training path.
- The app can insert live rows into traffic_logs.
- Data is shaped to location_name, current_speed, speed_ratio, and raw_data fields.

17.4 Rate Limits & Retries

There is no formal retry policy or backoff schedule at present. The system currently expects external APIs to be stable and fails gracefully on error rather than retrying automatically — tracked as a short-term roadmap item.

18. Deployment Architecture & Guide

Masar deploys as a single Docker container that serves both the static frontend and the JSON API from one Python process, which keeps the deployment surface small and avoids cross-origin complexity between a separate frontend and backend.

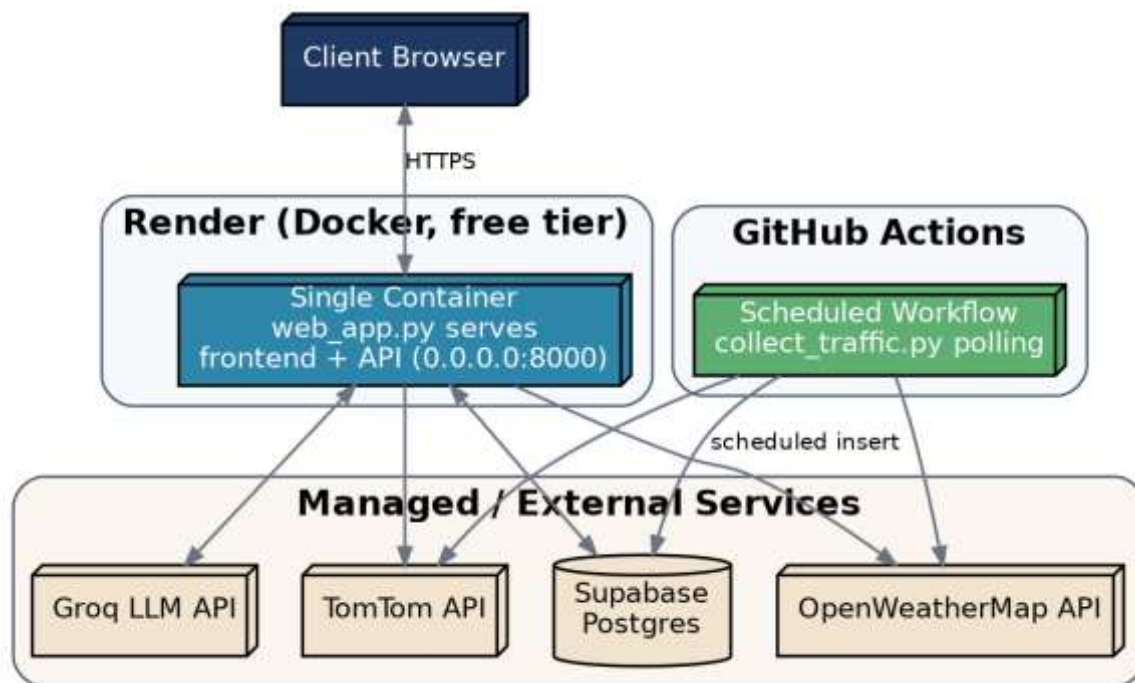


Figure 18.1 — Deployment topology: Render container, GitHub Actions, and external services.

18.1 Local Development

```
python -m venv VE
source VE/bin/activate
pip install -r requirements.txt
python train_model.py --synthetic-rows 6000
python web_app.py
# Open http://127.0.0.1:8000
```

On Windows PowerShell:

```
.\VE\Scripts\Activate.ps1
python train_model.py --synthetic-rows 6000
python web_app.py
```

18.2 Docker

```
docker build -t masar .
docker run --rm -p 8000:8000 --env-file .env masar
# or:
docker compose up --build
```

The Dockerfile builds a Linux image and runs the web app on port 8000. The server must bind to 0.0.0.0 rather than 127.0.0.1 (loopback) for external traffic to reach the container — a real bug caught and fixed during development.

18.3 Production Guidance

- Use a real secret store instead of .env files.
- Add ingress authentication and a reverse proxy.
- Persist model artifacts in a managed artifact store.
- Add health checks and deployment rollback automation.
- Consider a proper service layer if the project expands beyond a single app server.

19. Execution Lifecycle

19.1 Startup Sequence

6. Load environment variables with python-dotenv.
7. Initialize the standard-library HTTP server in web_app.py.
8. Determine bind address and port.
9. Serve the root page and API routes.
10. Optionally initialize a live traffic lookup if credentials are available.

19.2 Inference Sequence

11. Receive JSON request.
12. Build MasarState from the payload.
13. Refresh live fields if use_live_traffic is enabled.
14. Invoke build_graph().
15. Run predictor_node().
16. Route through router_node().
17. Optionally call context_node().
18. Run route_optimizer_node().
19. Call synthesis_node().
20. Return the bilingual result JSON.

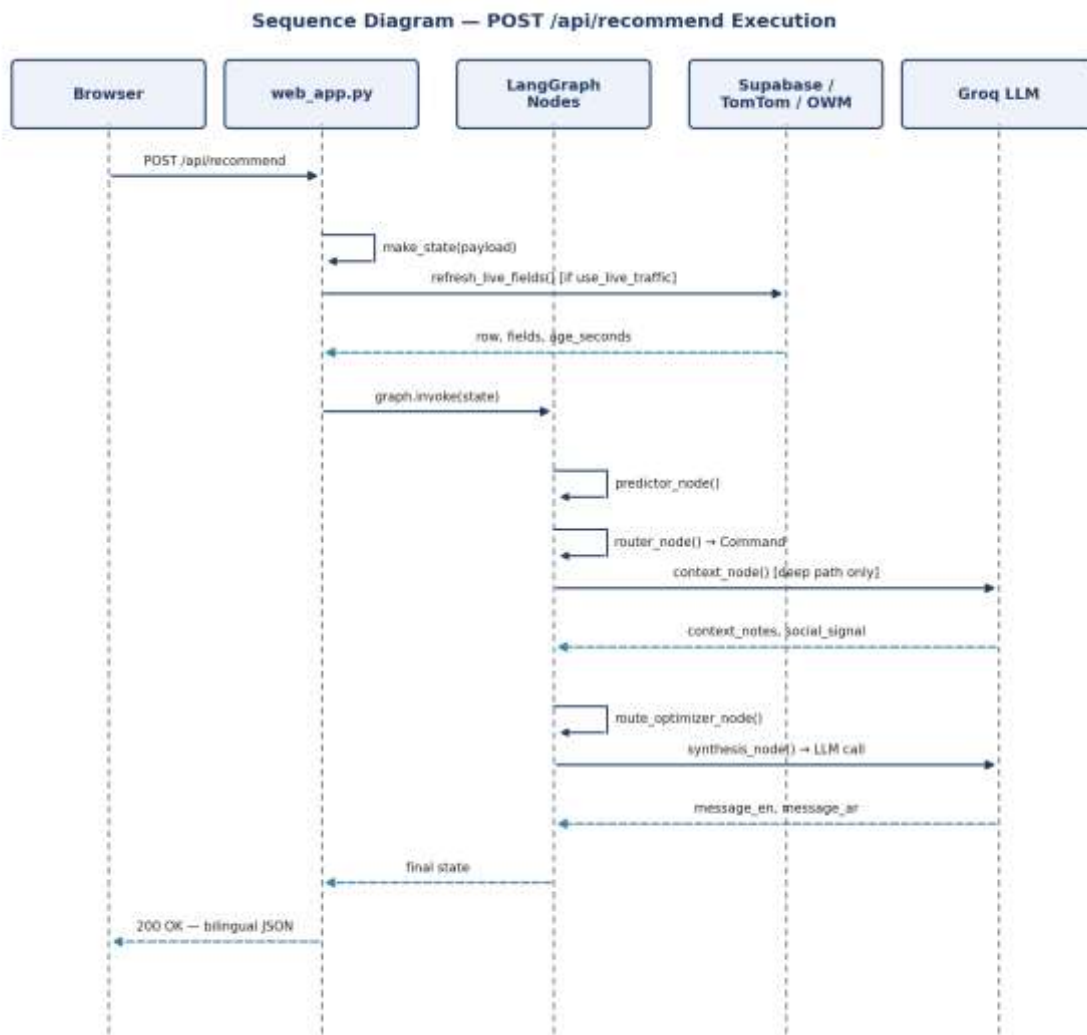


Figure 19.1 — Sequence diagram for POST /api/recommend.



Figure 19.2 — State transition diagram for a single request's lifecycle.

20. Performance Analysis

20.1 Built-in Observability

The project includes optional API call logging via `masar/api_report.py`, which redacts secret-like keys, records call metadata as JSON Lines, and writes to a path controlled by `MASAR_API_REPORT_PATH`.

- Latency per external call
- Request payload metadata
- Status (success / error)
- Response summary
- Error string, if present

20.2 Current Monitoring Gaps

- No centralized metrics backend.
- No Prometheus or Grafana integration.
- No explicit trace IDs beyond the local report file.
- No alerting thresholds defined in code.

20.3 Recommended Future Monitoring

- p95 latency for Groq / TomTom / OpenWeatherMap calls
- Request volume and failure rate per endpoint
- Model confidence distribution
- Route recommendation coverage and fallback counts

20.4 Performance Bottlenecks & Scaling Limits

Area	Current State
LLM calls	Dominate latency on the deep path
Live provider calls	Add network latency to every recommendation
Server model	Single-threaded-style HTTP process (standard library), not an async ASGI app
Horizontal scaling	Not implemented
DB connection pooling / orchestration	Not implemented
Asynchronous task queue	Not implemented

21. Error Handling Strategy

Error handling follows a consistent “degrade, don't crash” philosophy: every node has a documented fallback so a missing artifact, credential, or route never prevents a response from being returned.

Layer	Strategy
Predictor	Falls back to heuristic scoring when models/congestion_xgb.pkl is absent
Router	Treats missing/undefined confidence as an anomaly → deep path
Live data	Fails quietly or reverts to manual input when credentials are missing
Route optimizer	Returns empty route data (with a mode recommendation) when no path exists
API layer	Re-runs the graph in demo mode as a fallback if the regular run fails; returns 200/400/404/500/503 as appropriate

22. Security & Compliance

This is a local prototype and does not implement production identity or access control. It should be treated as demo-grade until the items below are addressed.

Area	Current State
Authentication	None for the HTTP server
Authorization	None
RBAC	Not implemented
Encryption in transit	Assumed via external providers' HTTPS
Secrets handling	.env-based local environment variables
PII handling	Minimal — no explicit PII collection
Audit logs	Basic redacted API report logs
Prompt injection defense	Not formalized

22.1 Data Protection Recommendations

- Put credentials in a secure secret manager in production.
- Add request authentication for `/api/recommend`.
- Avoid exposing raw provider payloads outside controlled logs.
- Add rate limiting and request validation for inbound traffic.
- Keep benchmark and report files local even though credentials are already redacted.

22.2 Compliance Notes

The repository does not yet declare GDPR, HIPAA, or SOC 2 controls. Use of external APIs should be reviewed against local data-retention and privacy-policy requirements before any production launch.

23. Current Limitations

23.1 Technical Limitations

- No formal production authentication or user identity model.
- Model and route logic are intentionally simple and explainable rather than enterprise-grade.
- The route graph is not an official transportation topology and should not be treated as authoritative navigation geometry.
- Social and event signals are stubs rather than production integrations.
- No robust retries, exponential backoff, or queueing for provider calls.

23.2 Data Limitations

- Real data may be sparse in early deployment states.
- The anomaly definition depends on throughput history and location-specific baselines.
- Location mapping is small and curated rather than a full city-wide transport model.

23.3 Known Assumptions

- Real-world commute analysis is approximated by a local graph and a heuristic route model.
- Weather and traffic are treated as the dominant factors, while broader social context remains intentionally limited.

23.4 Known Gaps (Tracked Deliberately)

Gap	Detail
Reddit social signal	Paused — the public JSON endpoint is blocked; PRAW + OAuth is the known fix, and credential scaffolding already exists in CI
Route Optimizer topology	Hand-built placeholder graph (five monitored corridors), not real RTA network data
Real-only model evaluation	Needs independently labeled future traffic outcomes; current evaluation uses synthetic labels
Reddit collection endpoint	Currently targets the public JSON endpoint despite OAuth secrets already being present in the workflow
<code>_get_social_signal /</code> <code>_get_nearby_events</code>	Hardcoded stubs in <code>context.py</code> , not yet wired to a live service

24. Future Roadmap

24.1 Short Term

- Complete the XGBoost training pipeline: finish `data_cleaning.py`, feature engineering, per-slot anomaly labeling, chronological train/val/test split with train-only baseline freeze, early-stopping training, model persistence + training report JSON, and a scheduled retrain CI workflow with a regression gate.
- Add request validation and stronger API error handling.
- Add retry logic for TomTom / OpenWeatherMap / Groq calls.
- Freeze exact dependency versions in `requirements.txt` for reproducible deployments.
- Expand tests for failure paths and rate-limit edge cases.
- Add an optional authentication layer for `/api/recommend`.

24.2 Medium Term

- Replace the corridor approximation with a richer route network (real RTA topology).
- Implement PRAW-based Reddit integration and wire it into the social-signal context.
- Support better model monitoring and drift analysis.
- Add a lightweight model registry and versioning strategy.
- Add a richer frontend dashboard with route comparison and time-series charts.

24.3 Long Term

- Multi-agent planning with additional specialized expert nodes.
- Better retrieval and memory systems for context-rich routing.
- Distributed inference and asynchronous model serving.
- Multimodal support for map imagery and route metadata.
- Stronger MLOps workflow with CI/CD, experiment tracking, and artifact governance.
- AWS deployment path spun up ahead of interviews (Render remains the always-on portfolio deployment).

NOTE

A parallel Autonomous Research Agent project (Search → Paper → Analysis → Synthesis agents, using a supervisor/orchestrator-worker pattern with conditional edges and cycles in LangGraph) is planned as a second, complementary LangGraph portfolio piece alongside Masar.

25. Testing & Quality Assurance

The repository contains unit tests and integration-style tests across the graph and web layer. Tests must be run with pytest from the repository root — running a test file directly with python bypasses pytest's path setup and raises `ModuleNotFoundError: No module named 'masar'`.

Test File	Purpose
tests/test_predictor.py	Model outputs, probability bounds, fallback heuristic, congestion ordering, state immutability
tests/test_router.py	Low, high, and missing-confidence routing
tests/test_context.py	Context component output using an injected fake LLM
tests/test_optimizer.py	Route calculation, mode thresholds, unknown endpoints
tests/test_synthesis.py	English and Arabic synthesis output
tests/test_graph.py	Full LangGraph deep and fast paths
tests/test_live_traffic.py	Supabase row mapping, rain/temperature handling, row age
tests/test_web_app.py	HTTP endpoint behavior
tests/test_training_data.py	Real/synthetic provenance and read-only training boundary
tests/test_export_synthetic.py	Synthetic export logic
tests/test_api_report.py	API report fields, response capture, secret redaction

25.1 Running Tests

```
python -m pytest tests/ -q

# Per-file (PowerShell / venv):
.\VE\Scripts\python.exe -m pytest tests\test_predictor.py -q
.\VE\Scripts\python.exe -m pytest tests\test_graph.py -q
```

25.2 Component-Level Verification (No Network Calls)

```
.\VE\Scripts\python.exe -m scripts.check_components --report-path reports/api_calls.jsonl
```

This prints PASS/FAIL results for every component and records the same run as JSON Lines. Each record includes a UTC timestamp, service, operation, sanitized request, returned response, duration, status, and error. Groq prompts/responses, offline LLM calls, and component function checks are included; API keys, tokens, passwords, and authorization values are redacted, and long values are truncated.

For an explicit Supabase read-only check, add `--include-network`:

```
.\VE\Scripts\python.exe -m scripts.check_components --include-network --report-path
reports/api_calls.jsonl
```

25.3 Coverage Expectations

The suite validates component-level behavior and graph-level flow. It is not a complete load or security test suite and should be extended for production readiness (see Section 24).

25.4 A Working Principle: Isolate Before Wiring

STATUS

Testing each node in isolation before graph wiring has already caught two real bugs in this codebase: the Router's fail-safe direction was inverted, and a Python import-by-value binding issue caused monkeypatching `CONFIDENCE_THRESHOLD` to silently fail (fixed by referencing the module rather than the imported name). This is treated as a standing engineering practice for the project, not a one-off fix.

26. Troubleshooting Guide

Fifteen issues that are either currently present in the project or are documented as likely failure modes, each with its root cause and resolution.

#	Issue	Root Cause	Resolution
1	Predictor confidence is always 0.4	models/congestion_xgb.pkl does not exist yet; predictor.py permanently uses the heuristic fallback	Complete the XGBoost training pipeline (Section 24.1) and bake the model at Docker build time
2	ModuleNotFoundError: No module named 'masar'	Running a test file directly with python tests/test_x.py bypasses pytest's path setup	Always run python -m pytest tests/ -q from the repository root
3	Container unreachable from outside	Server bound to 127.0.0.1 (loopback) instead of 0.0.0.0	Set MASAR_HOST=0.0.0.0 in the container environment
4	monkeypatching CONFIDENCE_THRESHOLD has no effect in tests	Python import-by-value binding — the constant was imported by name rather than referenced via the module	Reference the module attribute (e.g., router.CONFIDENCE_THRESHOLD) instead of importing the bare constant
5	Router always takes the deep path	prediction_confidence missing or undefined upstream	Verify predictor_node() is populating prediction_confidence before router_node() runs; missing confidence is a safe-by-design fail-open to the deep path
6	Live traffic never updates the state	Missing or invalid TOMTOM_API_KEY / OPENWEATHER_API_KEY / SUPABASE_* credentials	Confirm all four credentials are set; the app falls back to manual input silently when absent
7	route_optimizer_node() returns an empty route	No path exists between origin and destination in the hand-built corridor graph	Expected for districts outside the five monitored corridors; expand the NetworkX graph or supply a supported origin/destination pair
8	Reddit / social signal collection fails	collect_chatter.py targets a public JSON endpoint that is blocked	Switch to PRAW + OAuth; credential scaffolding already exists in the CI workflow
9	Training fails with 'dataset lacks both classes'	Synthetic or real sample is too small or too uniform to contain any anomalies	Increase --synthetic-rows, or include real data with --include-real
10	Training metrics look unrealistically strong	Benchmark uses synthetic labels generated by the same z-score rule the model is asked to learn	Treat synthetic benchmarks as a reproduction check, not real-world accuracy; report real-only metrics once independently labeled outcomes exist
11	Retraining on real data alone produces poor per-location baselines	Real Supabase coverage is currently ~81 rows (~1.5 days) — too sparse for a meaningful per-location/hour baseline	Accumulate at least 2–3 weeks of real coverage before relying on --real-only training
12	pip install fails with a system package conflict (e.g., PyJWT)	System-managed Python environment conflicts with pip-installed packages	pip install --break-system-packages --ignore-installed <package>

#	Issue	Root Cause	Resolution
13	bare pytest -q fails from a clean checkout	Some scripts execute code at import time outside the tests/ scope	Scope pytest to the tests/ directory: python -m pytest tests/ -q
14	Groq / synthesis call fails outside demo mode	Missing or invalid GROQ_API_KEY, or a provider-side outage	Set demo=true to bypass the live LLM call, or verify GROQ_API_KEY and provider status
15	API report file grows unexpectedly large or exposes prompt content	MASAR_API_REPORT_PATH logging captures prompts and responses (with secrets redacted, but not full payload content)	Keep reports/ local and out of version control; rotate or prune the JSONL file periodically

27. Appendix A — Facts, Assumptions & Recommendations

A.1 Facts

- The application is a local Python project built around a decision graph.
- It uses a trained XGBoost classifier and a fallback heuristic.
- It relies on Groq LLMs for context and synthesis.
- It reads live TomTom and OpenWeatherMap data when credentials are available.
- It exposes a simple HTTP API using the Python standard library.

A.2 Assumptions

- The project is currently a prototype rather than a production-grade mobility platform.
- Real RTA routes and official transport feeds are not yet integrated.
- Social and events data are not fully wired to production services.
- The route graph is an approximation of urban corridors, not a full city-network model.

A.3 Recommendations

21. Add a persistent secret manager for production deployment.
22. Replace the synthetic/minimal graph with a verified city network and a long-lived data pipeline.
23. Add formal MLOps tooling such as experiment tracking and a model registry.
24. Add stronger authentication and observability before any customer-facing launch.
25. Expand tests to cover failure modes, rate limits, and degraded network conditions.

28. Appendix B — Glossary of Terms

Term	Definition
LangGraph	A state-machine / graph orchestration framework used to wire together the predictor, router, context, optimizer, and synthesis nodes into a single stateful workflow.
MasarState	The shared TypedDict contract that flows through every node in the graph, carrying trip inputs and accumulating outputs.
Node	A single function in the LangGraph workflow that reads part of the state and returns updates to it.
Command	A LangGraph object returned by router_node() that both updates state and selects the next node to execute.
XGBoost / XGBClassifier	A gradient-boosted decision tree library used as the congestion anomaly classifier.
Heuristic fallback	A simple rule-based congestion estimate used when no trained model artifact is available.
NetworkX	A Python graph library used to model Dubai corridors and compute shortest paths for route optimization.
Confidence threshold	The 0.7 cutoff used by the router to decide between the fast path and the deep (context-enriched) path.
Fast path	The graph path taken when prediction confidence is high: predictor → router → optimizer → synthesis.
Deep path	The graph path taken when prediction confidence is low: predictor → router → context agent → optimizer → synthesis.
Synthesis node	The final node that converts route/context state into a bilingual (English/Arabic) message, validated by Pydantic.
Groq	The LLM inference provider used for the context and synthesis nodes.
Supabase	A managed Postgres service used to persist and query traffic_logs and related tables.
TomTom Flow API	The traffic-data provider supplying current and free-flow speed per segment.
OpenWeatherMap	The weather-data provider supplying temperature and rainfall.
FEATURE_ORDER	The single source-of-truth ordered list of model input features, shared by the predictor and the trainer.
Label leakage	A modeling error where information from outside the training window influences the labels; avoided here via a chronological split.
Demo mode	A configuration flag that bypasses live LLM/network calls so the app can run and be tested offline.
api_report.py	The module responsible for redacted, structured logging of external API and model calls.
RTA	Dubai's Roads and Transport Authority — the source of an official routing topology not yet integrated into this project.

29. Appendix C — Operational Command Reference

C.1 Training the Model

```
python -m scripts.train_model --synthetic-rows 2000 # synthetic only
python -m scripts.train_model --include-real        # + Supabase data
python -m scripts.train_model --real-only           # Supabase data only
```

C.2 Exporting Synthetic Data

```
.\VE\Scripts\python.exe -m scripts.export_synthetic --rows 6000
```

Creates data/synthetic_traffic.xlsx containing synthetic rows only (source=synthetic); never writes to Supabase.

C.3 Benchmarking the Model

```
.\VE\Scripts\python.exe -m scripts.benchmark_model `
--rows 6000 `
--model models\congestion_xgb.pkl `
--output reports\model_benchmark.json
```

Writes both reports/model_benchmark.json and a readable reports/model_benchmark.md, comparing the trained model against a majority baseline and the existing 1 – congestion_ratio heuristic (see Section 13.3 for current results).

C.4 Running Every Test File Separately

```
Get-ChildItem tests\test_*.py | ForEach-Object {
    Write-Host "===== $($_.Name) ====="
    .\VE\Scripts\python.exe -m pytest $_.FullName -q
    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
}
```

30. Document Summary

Masar is a compact, explainable, Dubai-focused mobility intelligence system that combines a tabular anomaly model, graph-based route optimization, and LLM-based reasoning. It is structured for local experimentation, demo deployment, and extension into a more production-grade commuter intelligence platform. The repository is intentionally simple and testable, with a clear separation between traffic data, graph logic, training, and user-facing recommendation output.

The system is best understood as a working prototype for urban congestion and multimodal route guidance rather than as a complete operational transportation system. With the training pipeline completed, real RTA topology integrated, and stronger observability and security controls in place, it is well positioned to become a substantially more robust product — while already standing on its own as a demonstration of applied multi-agent AI engineering.

End of Document